



UNIVERSITY OF MINNESOTA

Driven to Discover®

The Rendering Pipeline

CSCI 5980: Game Engine Architecture

Evan Suma Rosenberg | CSCI 5980 | Spring 2026

This course content is offered under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International license.



Today

- Overview of the rendering pipeline
- The tools and asset conditioning stages
- The GPU pipeline
- Programmable shaders
- Antialiasing
- The application stage
- Scene graphs

Rendering a 3D Scene

- A virtual scene is described, usually as 3D surfaces in some mathematical form
- A **virtual camera** is positioned and oriented to produce the desired view
 - Modeled as an idealized focal point with an imaging surface in front of it
 - Composed of virtual light sensors corresponding to **pixels** of the display
- **Light sources** are defined that provide all light rays interacting with and reflecting off objects in the environment
- **Surface visual properties** are described, defining how light interacts with each surface
- For each pixel, the rendering engine calculates the color and intensity of light converging on the camera's focal point through that pixel

Rendering a 3D Scene

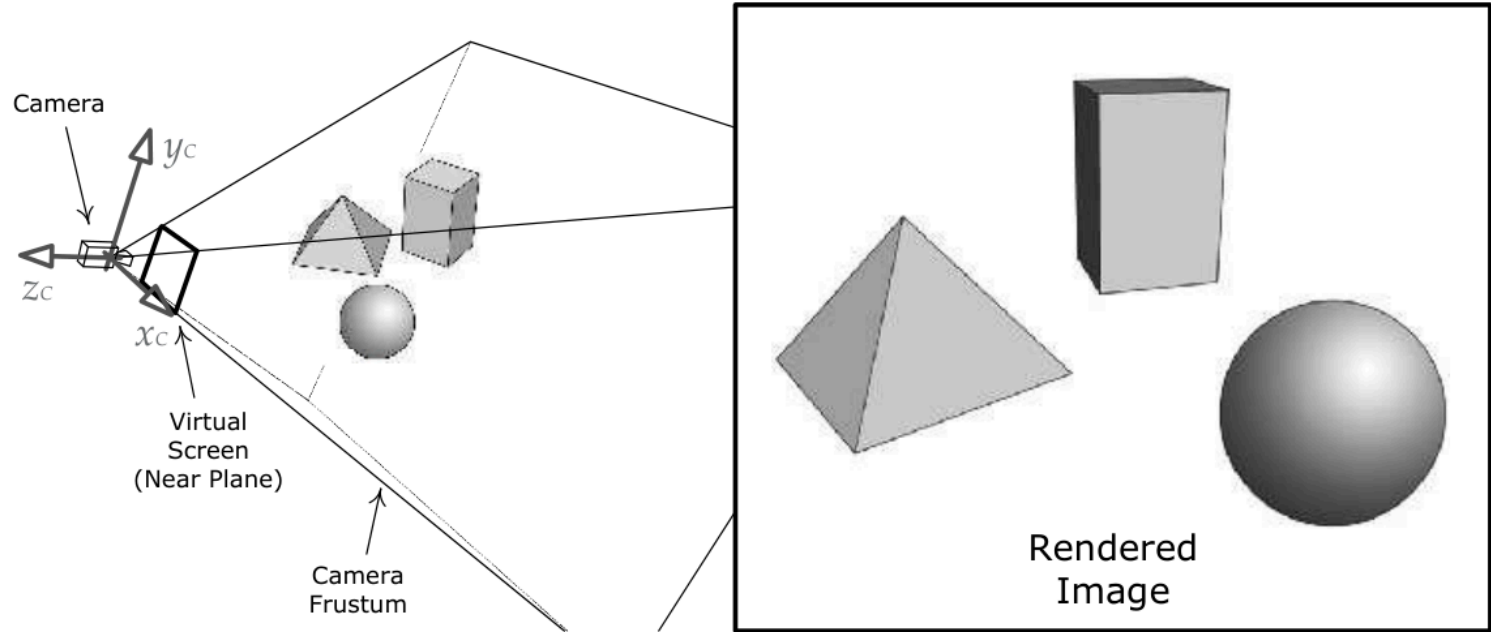
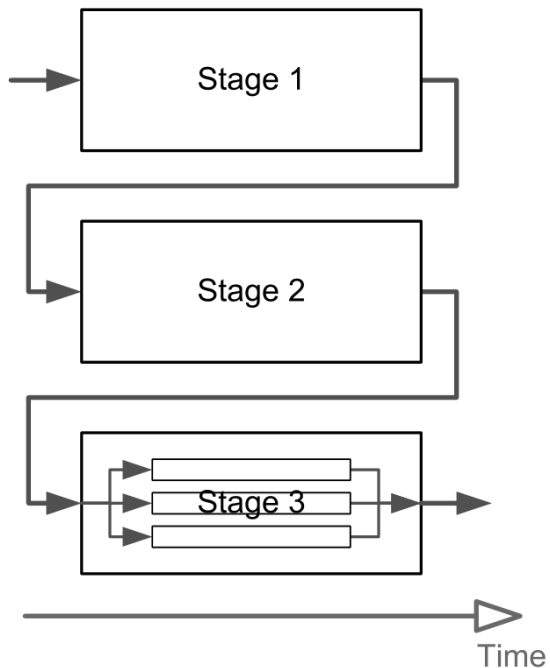


Figure 11.1. The high-level rendering approach used by virtually all 3D computer graphics technologies.

Pipeline Architecture

- A **pipeline** is an ordered chain of computational stages, each with a specific purpose
- Operates on a stream of input data items → produces an output stream
- Each stage can operate **independently** of the other stages
- Lends itself extremely well to **parallelization**
 - While stage 1 chews on one data element, stage 2 processes the previous result
 - Duplicate hardware N times → process N data elements in parallel



Pipeline Performance Metrics

- **Throughput**: How many data items are processed per second overall
- **Latency**: Time for a single data element to pass through the pipeline
- The **slowest stage** dictates the throughput of the entire pipeline
 - Also impacts the average latency as a whole
- Design goal: **minimize and balance** latency across the entire pipeline
- Eliminate **bottlenecks**
 - No stage should be idle for long waiting on another

Overview of the Rendering Pipeline

- Some texts divide the pipeline into three coarse-grained stages
- Gregory extends the pipeline to encompass offline tools as well
- **Five high-level stages:**
 - Tools stage (offline)
 - Asset conditioning stage (offline)
 - Application stage (CPU)
 - Geometry processing stage (GPU)
 - Rasterization stage (GPU)

Rendering Pipeline Stages

- **Tools stage (offline)**: Geometry and surface properties (materials) are defined
- **Asset conditioning stage (offline)**: Geometry and material data processed by the asset conditioning pipeline (ACP) into an engine-ready format
- **Application stage (CPU)**: Potentially visible mesh instances identified and submitted to the GPU along with their materials
- **Geometry processing stage (GPU)**: Vertices transformed, lit, and projected into homogeneous clip space; triangles processed by geometry shader, then clipped
- **Rasterization stage (GPU)**: Triangles converted into fragments that are shaded, tested (z, alpha, stencil), and blended into the frame buffer

How the Rendering Pipeline Transforms Data

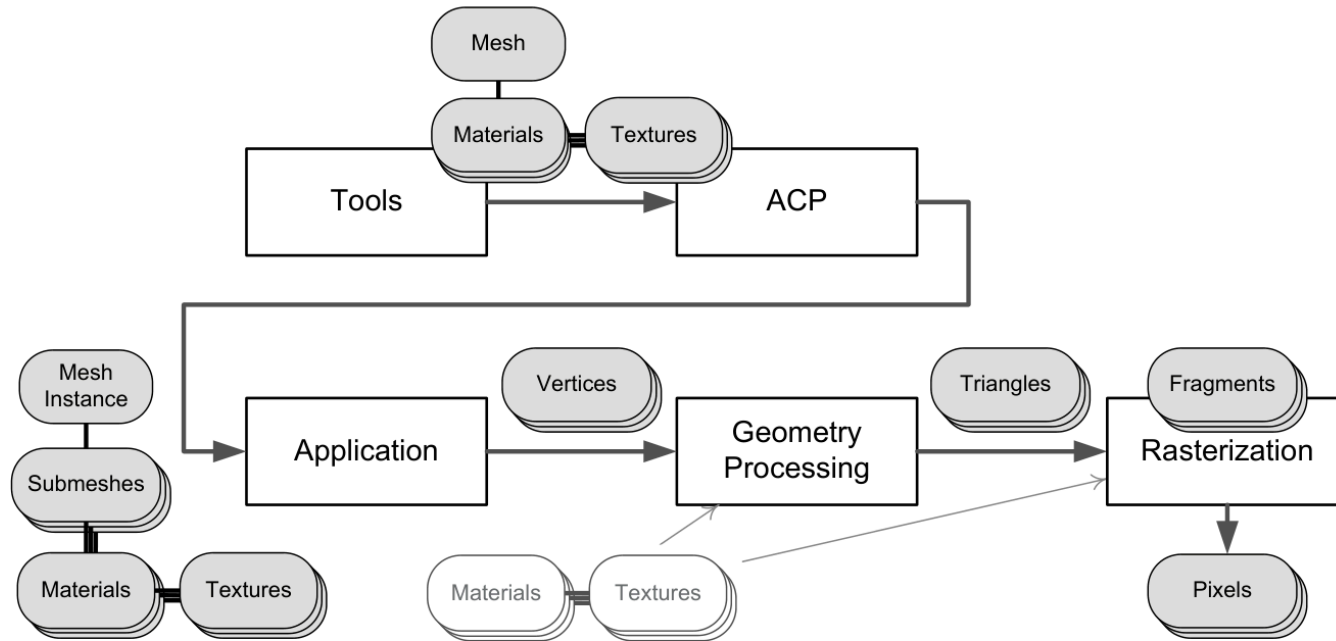


Figure 11.41. The format of geometric data changes radically as it passes through the various stages of the rendering pipeline.

The Tools Stage

- Meshes authored by 3D modelers in a **digital content creation (DCC)** application
 - Maya, 3ds Max, Lightwave, Softimage/XSI, SketchUp, etc.
- Models may use any convenient surface description
 - Triangles, quads, non-uniform rational basis splines (NURBS), etc.
 - Invariably **tessellated into triangles** prior to runtime rendering
- Vertices may be **skinned**: associated with one or more joints in a skeleton
 - Weights describe each joint's relative influence over the vertex
 - Used by the animation system to drive model movements

The Tools Stage: Materials

- Materials also defined by artists during the tools stage
- Involves selecting a **shader** for each material
- Selecting **textures** as required by the shader
- Specifying the **configuration parameters and options** of each shader
- Textures mapped onto surfaces
- Vertex attributes often "painted" with intuitive DCC tools

The Tools Stage: Material Editors

- Material editors may be:
 - Integrated directly into the DCC application as a **plug-in**
 - A **stand-alone** program
 - **Live-linked** to the game for real-time preview
 - Capable of shader authoring and debugging
- Example tools: NVIDIA Nsight Visual Studio Edition, Unreal Engine Material Editor
- Some tools use **node-based** WYSIWYG editors for rapid prototyping of visual effects

The Tools Stage: Material Libraries

- Materials stored with individual meshes can lead to **data duplication**
- Many games define a small number of reusable materials
 - Wood, rock, metal, plastic, cloth, skin, etc.
- Game teams build a **library of materials** from which to choose
- Individual meshes refer to materials in a **loosely coupled** manner

The Asset Conditioning Stage

- The ACP **exports, processes, and links** multiple asset types into a cohesive whole
- A 3D model comprises geometry (vertex/index buffers), materials, textures, optional skeleton
- The ACP ensures all referenced assets are available and ready to be loaded by the engine

ACP: Platform-Specific Processing

- Geometric and material data extracted from the DCC application
- Stored first in a **platform-independent intermediate format**
- Then processed into one or more **platform-specific formats**
- Ideally assets are ready to load into memory with **little or no postprocessing** at runtime
 - Example: Xbox One / PS4 mesh data output as GPU-ready index and vertex buffers
- ACP often considers **shader needs** when building assets
 - Example: Automatically generating tangent and bitangent vectors

ACP: Scene Graphs and Static Lighting

- **Scene graph data structures** may be computed during the ACP
 - Example: Static-level geometry processed to build a BSP tree
 - Helps quickly determine which objects to render for a given camera position
- **Static lighting** calculations often done offline:
 - "Baked" vertex lighting (light colors at mesh vertices)
 - **Light maps**: Texture maps encoding per-pixel lighting
 - **Precomputed radiance transfer (PRT)** coefficients

The GPU Pipeline

- Built around a specialized microprocessor: the **GPU**
- Designed to maximize **throughput** via massive parallelization
 - Example: AMD Radeon 7970 — 4 TFLOPS peak performance
 - 32 compute units × four 16-lane SIMD VPU × pipelined wavefronts of 64 threads
- GPUs are also fully programmable for **compute shaders**
 - Known as **general-purpose GPU computing (GPGPU)**

GPU Pipeline Substages

- GPU pipeline stages are categorized by their flexibility:
 - **Programmable** (full control via shader programs)
 - **Configurable** (fixed function with adjustable parameters)
 - **Fixed-function** (no customization)

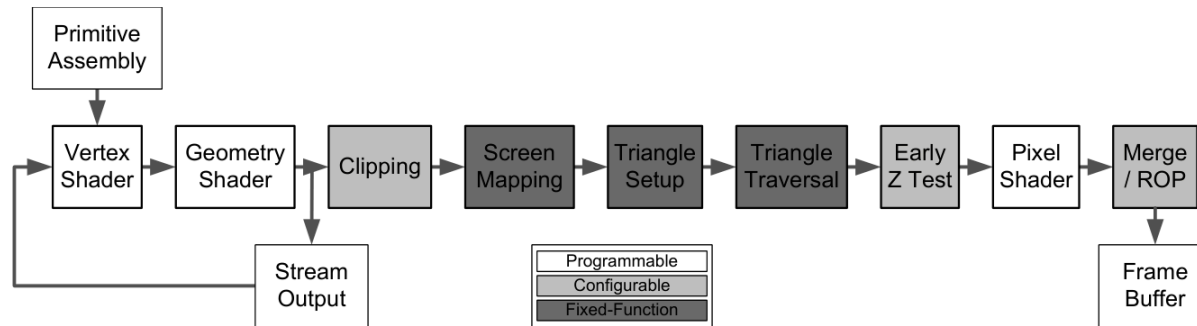
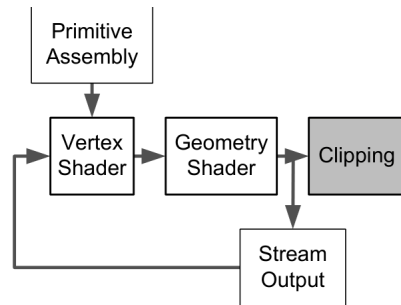


Figure 11.44. The geometry processing and rasterization stages of the rendering pipeline, as implemented by a typical GPU. The white stages are programmable, the light grey stages are configurable, and the dark grey boxes are fixed-function.

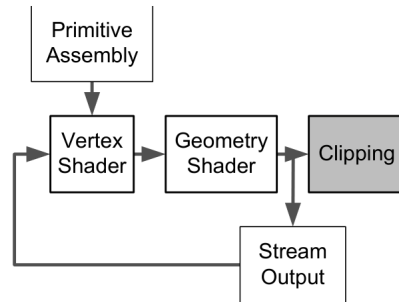
Vertex Shader

- Fully programmable
- Responsible for transformation and shading/lighting of individual vertices
- Input: Single vertex (position/normal in model space or world space)
- Operations performed:
 - Model space → view space via the **model-view transform**
 - **Perspective projection**
 - Per-vertex **lighting and texturing** calculations
 - **Skinning** for animated characters
 - **Procedural animation** (e.g., swaying foliage, undulating water)



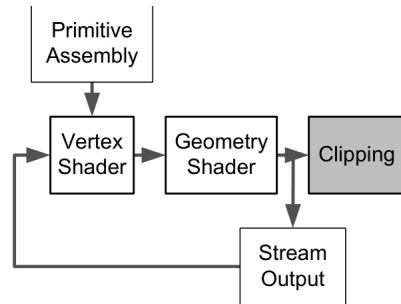
Geometry Shader

- **Optional** and **fully programmable**
- Operates on primitives (triangles, lines, points) in homogeneous clip space
- Can **cull**, **modify** input primitives, or **generate new** primitives
- Typical uses:
 - Shadow volume extrusion
 - Rendering six faces of a cube map
 - Fur fin extrusion around silhouette edges
 - Creation of particle quads from point data
 - Dynamic tessellation, fractal subdivision (lightning effects)
 - Cloth simulations



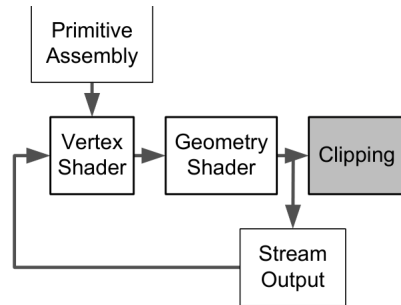
Stream Output

- Permits processed data to be **written back to memory**
- Data can then be **looped back** to the top of the pipeline for further processing
- Enables complex visual effects **without CPU involvement**
- Example: **Hair rendering**
 - Hair = collection of cubic spline curves
 - Vertex shader: Physics simulation on spline control points
 - Geometry shader: Tessellates splines into line segments
 - Stream output: Writes tessellated data to memory
 - Line segments piped back to top of pipeline for rendering



Clipping

- Chops off portions of triangles that **straddle the frustum**
- Identifies vertices outside the frustum
- Finds intersection of triangle edges with frustum planes
- Intersections become new vertices defining clipped triangles
- **Fixed in function**, but somewhat **configurable**:
 - User-defined clipping planes can be added
 - Can be configured to cull triangles entirely outside the frustum



Fixed Functions

- **Screen Mapping**

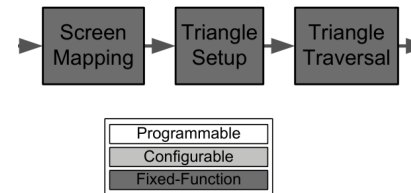
- Scales and shifts vertices from homogeneous clip space into **screen space**

- **Triangle Set-up**

- Initializes rasterization hardware for efficient triangle-to-fragment conversion

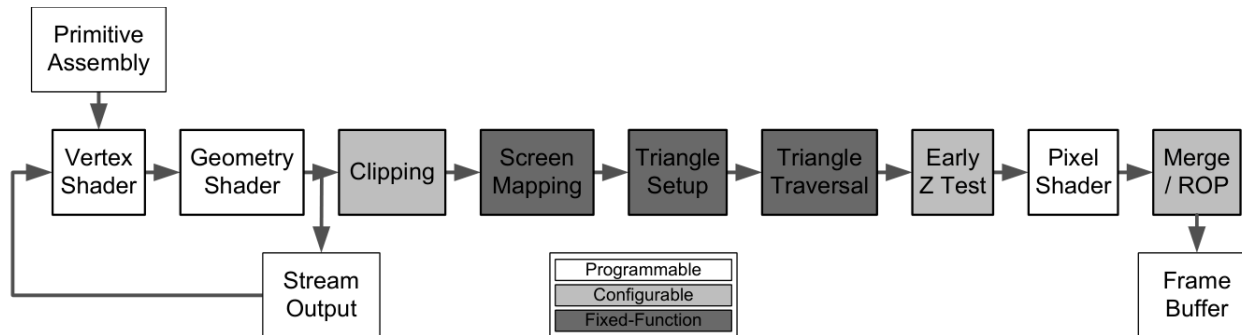
- **Triangle Traversal**

- Each triangle broken into **fragments** (i.e., rasterized)
- Usually one fragment per pixel; multiple with certain AA techniques
- **Interpolates vertex attributes** using perspective-correct interpolation



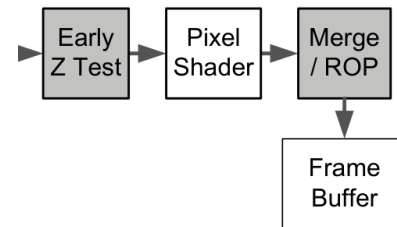
Early Z-Test

- Checks fragment **depth** at this point in the pipeline
- Discards fragments **occluded** by the pixel already in the frame buffer
- Allows the (potentially expensive) **pixel shader to be skipped** for occluded fragments
- Not all hardware supports this — older GPUs did the z-test **after** the pixel shader



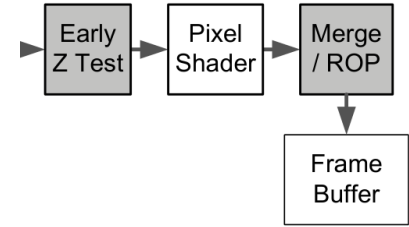
Pixel / Fragment Shader

- Fully programmable
- Shades (lights and processes) each fragment
- Can **discard fragments** (e.g., entirely transparent)
- Can address one or more **texture maps**
- Runs **per-pixel lighting** calculations
- Input: Per-fragment attributes (interpolated from vertex attributes by triangle traversal)
- Output: A single **color vector** for the fragment



Merging / Raster Operations Stage

- Also called the **ROP** (NVIDIA parlance)
- **Not programmable**, but **highly configurable**
- Runs various fragment tests:
 - **Depth test** (z-test)
 - **Alpha test** (reject fragments based on alpha channel values)
 - **Stencil test**
- If the fragment passes all tests, its color is **blended** with the existing frame buffer color
- Blending controlled by the **alpha blending function**



Programmable Shaders

- Shader architectures have evolved significantly since DirectX 8
- **Early shaders**: Low-level assembly only
 - Pixel and vertex shaders had different instruction/register sets
- **DirectX 9**: High-level C-like languages
 - **Cg** (C for graphics), **HLSL** (Microsoft), **GLSL** (OpenGL)
- **DirectX 10**: Introduced the **geometry shader**
 - **Unified shader architecture** (shader model 4.0)
 - All three shader types support roughly the same instruction set
 - All can read texture memory

Shader Input/Output Summary

- **Vertex Shader**

- Input: vertex in model/world space
- Output: transformed vertex in clip space

- **Geometry Shader**

- Input: single n-vertex primitive (+ optional control points)
- Output: zero or more primitives (possibly different type)

- **Pixel Shader**

- Input: fragment with interpolated attributes
- Output: color for frame buffer (or discard)

Accessing Memory

- Shader programs usually **cannot read/write memory directly**
- Access limited to: **registers** and **texture maps**
- Exception: **Heterogeneous system architectures (HSA)**
 - Example: PS4/PS5
 - CPU and GPU share **hUMA** (heterogeneous unified memory architecture)
 - Shaders can be passed a **shader resource table (SRT)**
 - SRT is a pointer to a C/C++ struct readable by both CPU and GPU

GPU Registers

- All GPU registers are **128-bit SIMD** format
 - Four 32-bit float or int values (`float4`)
 - Can hold a 4D vector, RGBA color, or a single replicated scalar
 - Matrices via groups of 3–4 registers (`float4x4`)
 - Some GPUs support 16-bit **halfs** (`half4` , `half4x4`)

GPU Register Types

- **Input registers**
 - Set automatically by GPU
 - Contain vertex attributes or interpolated fragment attributes
- **Constant registers**
 - Set by the application; constant only from the shader's perspective
 - Contain model-view matrix, projection matrix, light params, etc.
- **Temporary registers**
 - Internal to the shader; store intermediate results
- **Output registers**
 - Filled by the shader and passed to next stage in the pipeline

Accessing Textures

- Shaders have direct **read-only** access to texture maps
- Addressed via **texture coordinates**, not absolute memory addresses
- GPU's **texture samplers** automatically filter data
 - Blending between adjacent texels or mipmap levels
 - Filtering can be **disabled** for direct texel access (useful for data tables)
- Shaders can only **write** to textures indirectly
 - **Render to texture**: Render to an off-screen buffer, interpret it as a texture in subsequent passes

Shader Languages

- Languages like Cg, HLSL, and GLSL are modeled after **C**
- Declare functions, simple structs, perform arithmetic
- Variables and structs map directly onto **registers** via the shader compiler
- Mappings defined by:
 - **Semantics**: Bind variables to vertex/fragment attributes
 - **Input vs. output**: Determined by context (argument = input; return value = output)
 - **Uniform declaration**: `uniform` keyword accesses constant registers

Effect Files

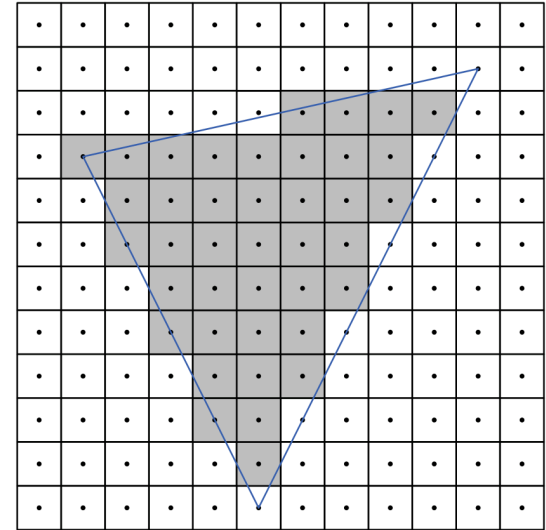
- A shader program alone isn't sufficient
- Additional information is required to call the shader program with meaningful inputs
- Must specify how application parameters map to uniform variables
 - Model-view matrix, projection matrix, light parameters, etc.
- Some effects require **multiple rendering passes**
- PC games may need **fallback** versions for older graphics hardware
- **Effect files** tie shader programs into a complete visual effect

Effect File Structure

- Various implementations: CgFX, OGRE material files, COLLADA/XML for GLSL
- **General hierarchical format:**
 - **Global scope:** Structs, shader programs, global variables
 - **Techniques:** One way to render a visual effect
 - Primary technique (highest quality)
 - Fallback techniques for weaker hardware
 - **Passes:** A single full-frame rendering pass within a technique
 - References to vertex/geometry/pixel shader "main" functions
 - Parameter bindings and optional render state settings

Antialiasing

- Rasterized triangle edges look **jagged**
- Aliasing arises from using a **discrete set of pixels** to sample a smooth, continuous 2D signal
- **Antialiasing** reduces these visual artifacts
- Net effect: "soften" edges by **blending** with surrounding pixels
- Each technique has unique performance, memory, and quality characteristics



Rastering a triangle without antialiasing

Full-Screen Antialiasing / FSAA

- Also known as **super-sampled antialiasing (SSAA)**
- Scene rendered into a frame buffer **larger** than the actual screen
- Resulting oversized image **downsampled** to the desired resolution
- **4× supersampling**: $2\times$ width $\times$ $2\times$ height
 - **4× the memory** and **4× the GPU processing**
 - Pixel shader runs four times per screen pixel
- Incredibly expensive and **rarely used** in practice

Antialiasing Techniques

- Practical antialiasing techniques include:
 - Multisampled Antialiasing (MSAA)
 - Coverage Sample Antialiasing (CSAA)
 - Morphological Antialiasing (MLAA)
 - Subpixel Morphological Antialiasing (SMAA)
 - Fast Approximate Antialiasing (FXAA)
 - Temporal Antialiasing (TAA)
 - Deep Learning Anti-Aliasing (DLAA)
- **TAA** is the most widely used technique in modern game engines
 - Further reading: Temporal Anti-Aliasing: A Blessing or a Curse?



Source: Digital Foundry

Let's Go Back to the Application Stage

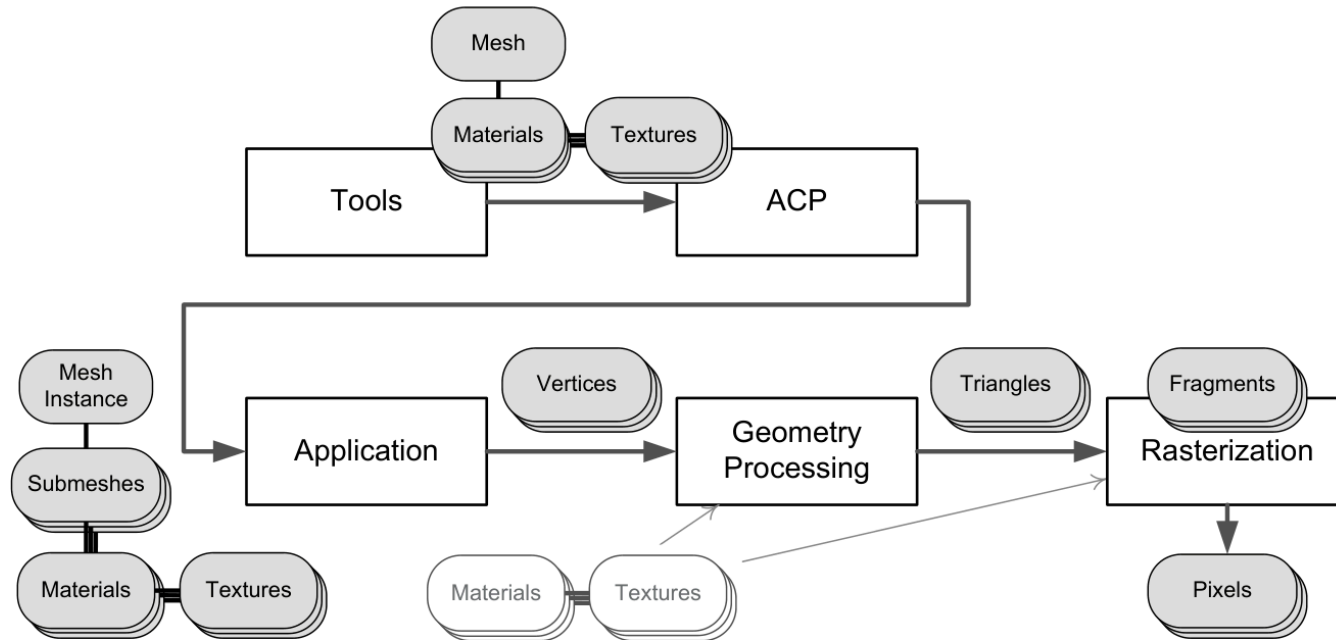


Figure 11.41. The format of geometric data changes radically as it passes through the various stages of the rendering pipeline.

The Application Stage

The application stage has **three roles**:

1. Visibility determination

- Only visible (or potentially visible) objects submitted to the GPU

2. Submitting geometry

- Sent via calls like `DrawIndexedPrimitive()` (DirectX) or `glDrawArrays()` (OpenGL)
- May be sorted for optimal performance; may be submitted more than once for multi-pass

3. Controlling shader parameters and render state:

- Configure uniform parameters and non-programmable stage settings per-primitive

Visibility Determination

- The **cheapest triangles** are the ones you never draw
- Important to **cull** objects from the scene that do not contribute to the rendered image
- Techniques include:
 - frustum culling
 - occlusion culling
 - potentially visible sets (PVS)
 - portals
 - antiportals

Frustum Culling

- All objects entirely **outside the frustum** are excluded from the render list
- Test each mesh instance's **bounding volume** against the six frustum planes
 - Bounding volume is usually a **sphere** (particularly easy to cull)
- For each frustum plane, compute distance h from sphere center to the plane:
 - $h = ax + by + cz + d = \mathbf{n} \cdot P - \mathbf{n} \cdot P_0$
 - Compare h_i against the bounding sphere radius
 - See section 11.2.7.1 of your textbook for more details
- Sphere on the front side of **all six planes** → inside the frustum

Occlusion Culling and PVS

- Even within the frustum, objects may **occlude one another**
- **Occlusion culling** removes fully occluded objects from the visible list
 - Very important in crowded, ground-level environments
 - Cost may outweigh benefits in sparse or aerial-view scenes
- Gross occlusion culling may be performed in large-scale environments
 - Precompute a **potentially visible set** from a given vantage point
 - Errs on the side of **inclusion** rather than exclusion

PVS Implementation

- Chop the level into **regions**
- Each region has a list of **other regions visible** from inside it
- PVS can be manually specified or generated by an **automated offline tool**:
 - Renders scene from random vantage points within a region
 - Color-codes each region's geometry
 - Scans the frame buffer to tabulate visible region colors
- Tools typically allow **tweaking**
 - Manually placing vantage points or explicitly including/excluding regions

Portals

- Game world divided into **semiclosed regions** connected by holes (windows, doorways)
- Holes = **portals**, represented by bounding polygons
- Rendering process:
 1. Render the region containing the camera
 2. For each portal, extend a **frustum-like volume** from camera through portal edges
 3. Cull neighboring region contents against this portal volume
- Only visible geometry in adjacent regions gets rendered
- Best for **enclosed indoor environments** with limited openings

Portals

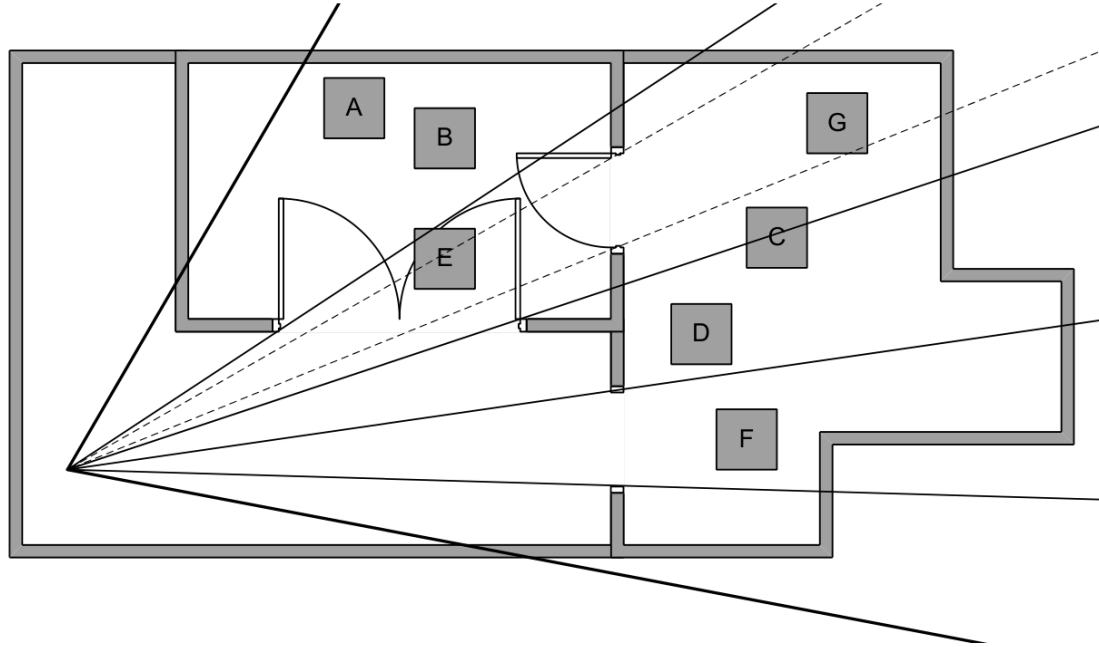


Figure 11.48. Portals are used to define frustum-like volumes, which are used to cull the contents of neighboring regions. In this example, objects A, B and D will be culled because they lie outside one of the portals; the other objects will be visible.

Occlusion Volumes (Antiportals)

- Pyramidal volumes describing regions that cannot be seen
- Construction:
 - Find **silhouette edges** of each occluding object
 - Extend planes from camera's focal point through these edges
 - Test distant objects against occlusion volumes
 - Cull objects entirely within the occlusion region
- Best for large outdoor environments with many nearby occluders
- **Portals** work best indoors; **antiportals** work best outdoors

Occlusion Volumes (Antiportals)

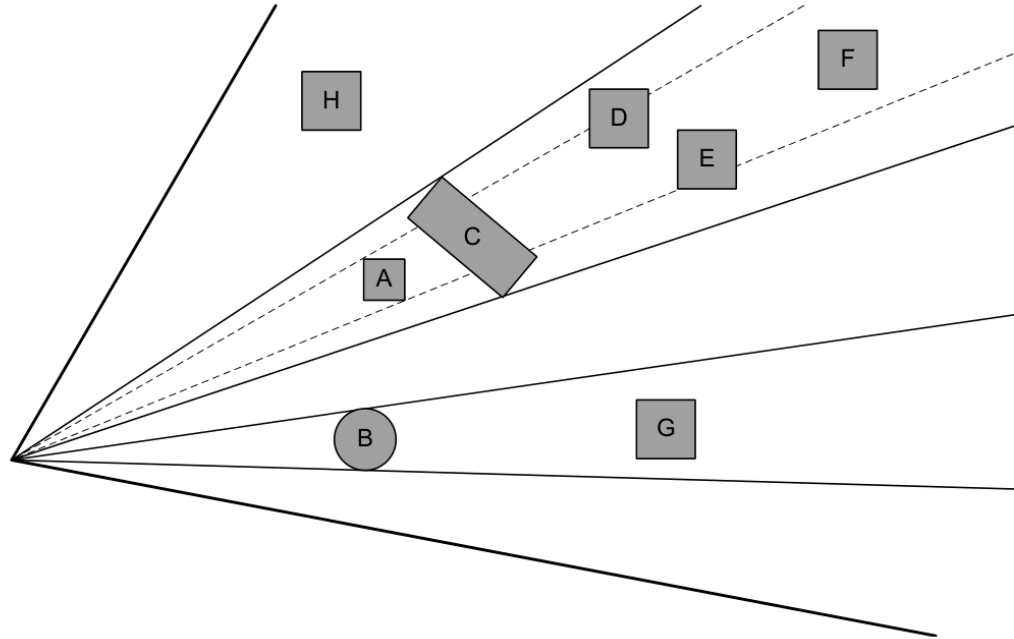


Figure 11.49. As a result of the antiportals corresponding to objects A, B and C, objects D, E, F and G are culled. Therefore, only A, B, C and H are visible.

Primitive Submission

- Visible primitives submitted via `DrawIndexedPrimitive()` (DirectX) or `glDrawArrays()` (OpenGL)
- **Render state**: The set of all configurable GPU pipeline parameters
 - World-view matrix, light directions, texture bindings
 - Texture addressing/filtering modes, time base for animation
 - Z-test enable/disable, alpha blending options
- Application stage must set render state **properly and completely** for each primitive

State Leaks

- If any render state is **not set** between submitted primitives, previous settings "leak" onto the new primitive
- May manifest as: wrong texture, incorrect lighting, etc.
- Critical that the application stage **never allows state leaks**

The GPU Command List

- Application communicates with GPU via a **command list**
 - Interleaves render state settings with geometry references
- Example: Objects A, B with material 1; then C, D, E with material 2:
 1. Set render state for material 1 (multiple commands)
 2. Submit primitive A
 3. Submit primitive B
 4. Set render state for material 2 (multiple commands)
 5. Submit primitive C, D, E
- API calls like `DrawIndexedPrimitive()` construct and submit these lists under the hood
- Some engines build command lists **manually** or use low-level APIs like **Vulkan**

Geometry Sorting

- Render state settings are **global**
 - Changing them requires flushing the entire GPU pipeline
- Goal: Change render settings **as infrequently as possible!**
- Solution: **Sort geometry by material**
 - Install material A settings → render all material A geometry → move to B
- Problem: Material sorting increases **overdraw**
 - Same pixel filled multiple times by overlapping triangles = wasted bandwidth

Front-to-Back vs. Material Sorting

- **Early z-test** discards occluded fragments before the expensive pixel shader runs
- To maximize early z benefit: Draw triangles in **front-to-back order**
 - Closest triangles fill the z-buffer first
 - More-distant fragments quickly discarded with minimal overdraw
- But this **conflicts** with sorting by material!

Z-Prepass to the Rescue

- Render the scene **twice!**
- **Pass 1: Z-prepass**
 - Generate z-buffer contents as efficiently as possible
 - GPU provides a special **double-speed mode** (pixel shaders disabled, only z-buffer updated)
 - Opaque geometry rendered **front-to-back**
- **Pass 2: Full color**
 - Populate frame buffer with color information
 - No overdraw thanks to the z-buffer
 - Geometry resorted into **material order** for minimal state changes

Transparent Geometry (2 methods)

- After opaque geometry rendered, transparent surfaces drawn in **back-to-front order**
 - This brute-force method allows proper alpha-blended results
- **Order-independent transparency (OIT):**
 - Permits transparent geometry to be rendered in **arbitrary order**
 - Stores **multiple fragments per pixel**, sorts them, blends after scene is rendered
 - Correct results without pre-sorting
 - High memory cost (frame buffer must store all translucent fragments per pixel)

Scene Graphs

- Modern game worlds can be **very large**
- Most geometry is outside the frustum
 - Culling every object explicitly is **wasteful**
- Need a data structure to:
 - Quickly **discard large swaths** of the world far from the frustum
 - Help **sort geometry** (front-to-back for z-prepass, material order for full-color)
- Often called a **"scene graph"**
 - Though usually a tree, which is a special case of a graph

Scene Graphs: Core Idea

- **Partition 3D space** so that regions not intersecting the frustum can be discarded without per-object culling
- Common spatial data structures:
 - Quadtrees and octrees
 - BSP trees
 - kd-trees
 - Bounding sphere trees
 - Spatial hashing techniques
- See section 11.2.7.4 of your textbook for technical details

Choosing a Scene Graph

- Choice depends on the nature of your scenes
- **Fighting game** (small arena): May not need much of a scene graph
- **Enclosed indoor environments**: BSP tree or portal system
- **Outdoor terrain from above** (strategy game): Simple quadtree
- **Outdoor from ground level**: May need additional culling
 - Dense scenes → antiportals benefit from many occluders
 - Sparse scenes → antiportals may hurt frame rate
- Base your choice on hard data from measuring actual rendering performance!

Key Takeaways

- The rendering pipeline is a **software/hardware architecture** with ordered stages optimized for parallelism
- The pipeline spans from **offline tools** (meshes, materials, ACP) to **runtime GPU stages** (geometry processing, rasterization)
- Data transforms through the pipeline
 - meshes → mesh instances → vertices → triangles → fragments → pixels
- The **slowest stage** determines pipeline throughput
 - Design for **balanced latency**

Key Takeaways

- The GPU pipeline mixes:
 - **programmable** stages (vertex, geometry, pixel shaders)
 - **configurable** stages (clipping, merging/ROP)
 - **fixed-function** stages
- Shaders access data through:
 - **registers** (input, constant, temporary, output)
 - **textures**
- **Effect files** tie shaders, parameters, and render state into complete visual effects with techniques and passes

Key Takeaways

- The application stage handles **visibility determination**, **primitive submission**, and **render state management**
- **Frustum culling**, **occlusion culling**, **PVS**, **portals**, and **antiportals** are key visibility strategies
- **Geometry sorting** involves a trade-off between material and front-to-back order
- **Scene graphs** partition space to accelerate frustum culling
- The right scene graph depends on your game; **measure performance** to decide



Questions?

No participation exercise today.